

What Environment Do LLM Agents Actually Learn In?

Ying Wen

Shanghai Jiao Tong University
Shanghai Innovation Institute

`ying.wen@sjtu.edu.cn`

May 6, 2026

Abstract

Learning from the execution of language agents requires a precise account of the environment in which observations, actions, feedback, and updates are defined. This paper proposes a working definition of a large language environment with five interfaces: the external world, the runtime, the internal task environment, feedback evidence, and update carriers. It uses coding and tool-use examples to distinguish intended actions from executed effects and raw task logs from reusable experience. The framework identifies learning targets at the representation, interaction, and update-routing levels, and describes the evidence needed to promote experience into memory, skills, training data, or parameters. It concludes with a research agenda for verifiable, transferable, and reversible experience reuse; the framework is a design proposal rather than a validated general learning algorithm.

Keywords: llm-agents, reinforcement-learning, large-language-environments, continual-learning

Start with a mundane example. A coding agent takes over an unfamiliar repository, reads an issue, edits files, runs tests, and fails. The failure is not surprising. The real question is what happens the second time it sees a similar repository. Should it remember the error stack, remember a project-specific configuration, abstract a debugging procedure, or write this experience into training data or even model weights?

That question comes before the question of whether to use reinforcement learning. LLM agents clearly need to keep improving from their own actions, failures, corrections, and verification results. Sutton and Barto’s classic reinforcement learning framework emphasizes that agents improve future decisions through interaction with an environment and feedback from consequences [1]. Silver and Sutton push this into a stronger research program: the next stage of agent capability may depend increasingly on streams of experience generated through continual interaction, not only on static human-written corpora [2].

But the consensus can jump too quickly. Many discussions begin by asking whether we should use GRPO [3], PPO, or directly train on tool-use logs as trajectories. GRPO and PPO may be useful backend optimizers, but they are not the right starting point for this essay. Reinforcement learning may be part of the answer. If it is, we must first ask: what environment is it learning in?

The argument proceeds in four steps. First, agent learning is framed as an environment-interface design problem. Second, we define a large language environment. Third, we explain how this definition reopens the objects usually called state, action, feedback, and update target in reinforcement learning. Fourth, we discuss how logs from one task can become verifiable, transferable, and reversible experience.

Figure 1 gives the basic contrast. Traditional reinforcement learning environments such as Go, StarCraft, or Honor of Kings can of course be extremely complex. A game engine or simulator is already an environment. The point is that it usually provides a relatively clear external

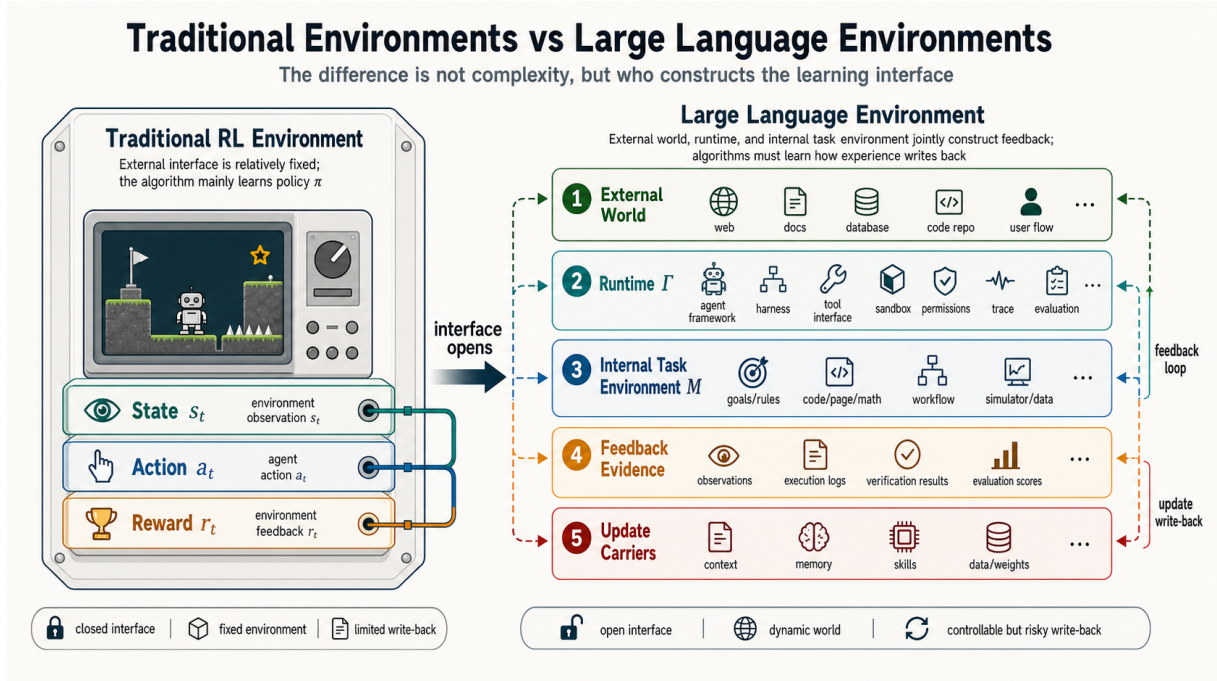


Figure 1: Core differences between traditional environments and large language environments

interface: how observations are produced, what actions are allowed, how transitions happen, and how win conditions, scores, or task rewards are computed. The hard part in large language environments is not simply that the external world is more complex. It is that state, action, feedback, and update targets are jointly constructed through the language runtime, tools, memory, verifiers, and the model’s internal task environment.

This also helps explain today’s long-thinking and planning abilities. Much of long thinking is extended search after the model already has a rough understanding of the task environment: what objects exist, what constraints cannot be violated, which intermediate steps can be checked, and what evidence can verify the final answer. In math problems, the evidence may be formal derivation and final equality. In coding tasks, it may be tests, compilation, and reproduced errors. In web tasks, it may be page state and user goals. Long thinking asks how to search inside an already somewhat clear internal task environment. Reinforcement learning in large language environments asks an earlier question: where did that internal task environment come from, what happens when it is wrong, and can real interaction evidence correct, store, and reuse it?

Figure 2 uses Monte Carlo tree search in Go and chess as the analogy. Traditional search works because it usually depends on a known or queryable exact environment: rules are known, transitions can be simulated, and win or loss can be verified. Long thinking, planning, and self-verification in large language environments also unfold internally first, but they depend on the model’s approximate imagination of the real external world and runtime. This approximation is what I call the internal task environment. In a more technical vocabulary, it is also close to a world model. It is not the external environment itself. It is the internal substitute the model uses to predict action consequences, check plans, and generate the next move.

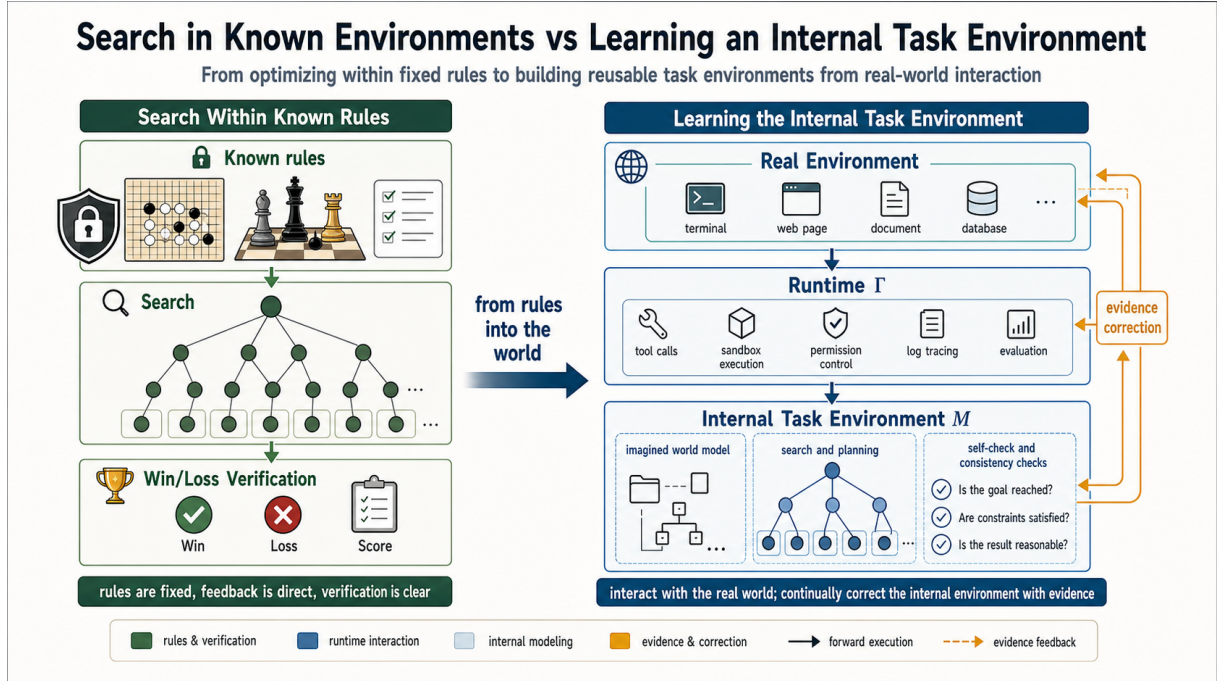


Figure 2: Search in known environments versus learning an internal task environment

1 First Ask the Right Question: We Are Designing a Learning Environment

The initial leap in large model capability came mainly from static data scaling. More text, more code, larger models, and larger training budgets gave models broad language, coding, and reasoning priors. In that stage, the central question was how to learn general capabilities from existing data.

The agent era changes the question. An agent does not merely answer. It acts. It opens files, runs commands, clicks web pages, calls APIs, writes code, waits for tests, receives review comments, reads logs, remembers user preferences, and reuses past procedures. Its ability comes not only from what it read before training, but also from what it experiences after deployment.

The point of this section is not to choose an optimizer immediately. It is to ask how the learning environment is designed. Use the coding agent as the running example. The first failed attempt on an unfamiliar repository may fail because the agent did not observe a key configuration, because its action did not actually land, because the tests were weak evidence, or because it mistook a temporary user preference for a long-term rule. Each failure points to a different learning site.

If the problem is that a key configuration was invisible, the observation construction should change. If the problem is that the model believed it had edited a file but the runtime did not have write permission, the action grounding should change. If the problem is that tests only covered a surface path, the feedback evidence and verification budget should change. If the problem is that a local experience was written as a general skill, the update routing should change. In other words, agent learning is not finished once we pour logs into a training algorithm. We first need to design the interfaces among state, action, feedback, and update carriers.

Standard reinforcement learning usually begins with an external environment whose interface has already been specified: what observations the agent receives, what actions it can execute, what feedback counts as reward, and when an episode ends. Traditional reinforcement learning has certainly studied partial observability, state abstraction, hierarchical action, reward

learning, and non-stationary environments. The new point in large language environments is not that those problems disappear. It is that once the external world is mediated through language runtimes, tools, memory, permissions, verifiers, and internal task environments, the interface itself becomes something the learning system must define, calibrate, and govern.

A better starting point is therefore not to select a training trick, but to define the agent’s learning environment: what it can observe, what it can affect, where feedback comes from, which experiences can be written into future systems, and which experiences must remain local to the current task.

2 Definition First: A Large Language Environment Has Five Interfaces

Here is the conclusion first. A large language environment is not a single simulator. It is a learning field jointly composed of the external world, runtime, internal task environment, feedback evidence, and update carriers. In symbols:

$$\mathcal{L}_t = (\mathcal{W}_t, \Gamma_t, \mathcal{J}_t, \mathcal{F}_t, \mathcal{U}_t)$$

Before caring about the letters, keep the plain-language map in mind:

Component	Intuitive meaning	Example in a coding agent
$\mathcal{W}$ external world	Objects that can actually be changed	Repository files, dependencies, terminal, test environment, issue
Γ runtime	The mediation layer that exposes the external world to the model	Agent framework, execution runtime, task harness, tool interface, permissions, sandbox, trajectory records
$\mathcal{J}$ internal task environment	The internal space the model uses for planning and prediction	Hypotheses about code structure, bug location, and test consequences
$\mathcal{F}$ feedback evidence	Calibrating signals received after action	Build results, unit-test failures, error stacks, review comments
$\mathcal{U}$ update carriers	Where experience is finally written	Current context, project rules, memory, skills, data, weights

$\mathcal{W}_t$ is the externally constructed environment. It includes the codebase, web pages, file system, terminal, APIs, graphical interfaces, users, and organizational processes. It is not a pure text environment. The real effects still happen in the external world: files change, tests pass, page state changes, calendar invites are sent, permissions are triggered.

Γ_t is the runtime environment. It is neither the external world itself nor model weights. Related framework, runtime, and harness distinctions appear in LangChain documentation [4]. It is the mediation layer that turns the external world into something visible, operable, and feedback-producing for the model. Two agents with identical model weights can face entirely different learning problems if their runtimes differ. One runtime may expose the full error stack while another only says “test failed”; one may execute shell commands while another has no terminal permission; one may preserve project memory while another clears state every turn.

From an engineering perspective, Γ_t is closer to the agent’s execution interface layer than to any single agent framework or evaluation script. Existing engineering materials usually split this layer more finely. The first category is the agent framework. It provides abstractions for models, tools, messages, structured outputs, and loop control so developers can assemble lan-

guage models into runnable agents. Handoffs, guardrails, sessions, and tracing may be provided by the framework, but not every framework provides them in the same way. The second category is the execution runtime. It handles state saving, recovery, streaming execution, human intervention, and persistence for long-running tasks; LangGraph documentation places such capabilities in the runtime layer. The third category is the agent harness: an opinionated shell on top of frameworks and runtimes, often preconfiguring planning, subagents, file systems, context compression, and long-task management.

Therefore, when these harnesses participate in observation construction, action grounding, environment reset, and evidence return during an interaction trajectory, they should be included in Γ_t . If they only score a task offline after the task ends, they are closer to the evidence and verification channel represented by $\mathcal{F}_t$ or V . Ignoring this layer during training creates runtime mismatch. The model may learn how to write text that looks like a solution rather than how to reliably observe, act, change the external world, and pass verification inside a specific runtime. Thus Γ_t in a large language environment is not merely glue that connects tools to the model. It is a core interface that defines the boundaries of state, action, feedback, and experience.

$\mathcal{J}_t$ is the internal task environment. It includes the context window, system instructions, retrieved memories, current plan, candidate reasoning paths, and the internal world model used by the model during long thinking and planning. This internal task environment is not necessarily just an approximation of the external tool world. It can also be the mathematical space, proof space, code abstraction space, legal-rule space, or organizational-process space learned by the model. It contains objects, constraints, executable reasoning transformations, local verification grounds, and failure modes.

Math tasks illustrate this point. A math problem usually has no queryable external environment like a Go engine. The prompt gives symbols, definitions, and constraints; the final answer may be checked by humans, formal verifiers, or a standard solution. But the place where the model actually searches is the mathematical space it has learned internally: which transformations are legal, which lemmas may help, which intermediate expressions are checkable, and which paths lead to contradiction. If reinforcement learning treats math as only problem input plus final-answer reward, it misses the key object: the model’s internal, language-mediated mathematical environment, which can be searched, self-checked, wrong, and corrected by feedback.

$\mathcal{F}_t$ is the feedback-evidence space. It includes tests, compilation, environment state changes, user corrections, review comments, safety rules, model judgments, and private evaluations. $\mathcal{U}_t$ is the space of update carriers: current context, short-term memory, long-term memory, project rules, skills, verifier examples, data assets, and model weights.

The difference between this definition and environments such as Go, StarCraft, or Honor of Kings is not that those environments are simple. StarCraft and Honor of Kings have large state spaces, action spaces, and multi-agent game dynamics. But they typically still expose a game-engine-defined external interface: how observations are produced, which actions are legal, how transitions execute, and how win conditions or scores are returned. In a large language environment, observation, action, feedback, and update targets are jointly constructed by $\mathcal{W}_t$, Γ_t , $\mathcal{J}_t$, $\mathcal{F}_t$, and $\mathcal{U}_t$.

From a continual-learning perspective, the whole large language environment can also be viewed as a persistent large world, or an environment family. Each concrete task is not an isolated environment that appears from nowhere. It is a micro-world cut out of the large world by the runtime: a code-fix micro-world, a web-booking micro-world, a math-proof micro-world, or an office-approval micro-world. A micro-world has its own goal, visible context, tool permissions, local verification methods, and internal task environment. The large world preserves the shared tool ecosystem, memory, skills, verifiers, organizational rules, and model weights across tasks.

This level distinction matters. Continual reinforcement learning is not unlimited trial and error inside a single task. It is the problem of deciding, across many micro-worlds, which experiences should be written back to the large world and which should remain local. A special repository configuration may belong only to the current project. A reliable error-reproduction procedure may become a cross-repository skill. Defining the large language environment is what lets the system distinguish the two.

Hint: define the environment before talking about reinforcement learning. For LLM agents, the environment definition must answer at least three questions: what external objects can be acted on, how the runtime exposes them to the model, and where the internal task environment used for long thinking, planning, and self-verification comes from, how it is corrected by evidence, and how it is updated.

3 Three Differences Between RL in Large Language Environments and Classical RL

The core of classical reinforcement learning remains valid: the agent acts, the environment gives feedback, and the agent uses experience to improve future behavior. The issue is not that this core has expired. The issue is that many traditional setups first wrap the environment interface, then optimize the policy inside that closed interface. This is powerful, but it can also turn the research problem into a kind of closed-door optimization: how the external world is observed, how actions really land, how feedback is verified, and how experience affects the future system are all flattened into already-given states, actions, and rewards.

For LLM agents, this flattening breaks down. The external world is not a simple state tensor, the runtime is not transparent air, and the model's internal task environment is not always reliable. When a coding agent says "I fixed the bug," the sentence itself does not mean the external files were correctly changed. When a web agent plans to click a button, the plan does not mean the runtime has found the right element and triggered the page transition. When a math agent thinks a proof is valid, that does not mean every transformation has a legal basis.

RL in large language environments should therefore be separated into at least three layers.

The first is the representation layer. Traditional environments usually provide states or observations directly, and action spaces often have clear boundaries. In large language environments, states, observations, and actions are larger, sparser, and more ambiguous. The state of a coding task may be distributed across requirements, repository files, dependencies, error stacks, test results, project conventions, and edit history. The state of a web task may be distributed across screenshots, DOM, login state, page transitions, and user goals. The state of a math task may be distributed across problem conditions, symbolic relations, known theorems, candidate intermediate expressions, and proof goals. The representation layer asks: what should be visible in the current micro-world, what should be abstracted, and what must be preserved as later verification evidence?

The second is the interaction layer. An LLM agent interacts with the external world, the runtime, and the internal task environment at the same time. External interaction happens in repositories, web pages, terminals, files, APIs, and human organizational processes. Internal interaction happens in the model's mathematical space, proof space, code abstraction space, planning tree, and self-verification process. Internal reasoning steps, external action requests, runtime execution, and actual environmental effects are not the same thing. The agent framework determines how the model organizes messages, generates requests, calls tools, and switches roles. The execution runtime and task harness determine whether a request actually executes, what external state it reaches, and what evidence is returned. If training does not separate these parts,

the system only knows that “the task failed.” It does not know whether to change observation, the internal task environment, tool parameters, permissions, or action grounding.

The third is the coupling layer. In large language environments, the same LLM often plays three roles at once: policy, internal world model, and reward or value judge. It proposes actions, predicts action consequences inside the internal task environment, evaluates answers, performs self-verification, and summarizes experience. A training update or memory update may therefore change not only the policy π , but also the internal task environment M and the value or reward judgment V . A system may change its self-verification standard while updating its policy; it may change its internal world model while updating memory; it may create self-confirming bias when the evaluator model and policy model share the same blind spots.

These three layers are not there to make the story complicated. They prevent three common mistakes: treating logs as experience, treating feedback as reward, and writing local success as global capability. A log can become experience only after task boundaries, action grounding, evidence verification, and update routing. Feedback should be interpreted as reward, preference, or constraint only after it carries source, scope, and anchor strength. Success in a micro-world should be written back to the larger system only after cross-task reproduction and risk checks.

The question is therefore not whether traditional reinforcement learning remains useful. The question is that the objects often stabilized in traditional settings, namely states, actions, rewards, and update targets, are reopened in LLM agents. Reinforcement learning cannot only ask how to optimize a policy. It must also ask how states and actions are represented, how internal and external environments interact, and how policy, internal environment model, and value judgment are jointly updated without contaminating one another. It must also ask how experience from a micro-world is filtered, abstracted, and written back so the whole large world becomes more reliable with use.

4 Formalization: What Should the Algorithm Learn?

The definition above says what the environment is. The next question is what the algorithm should learn in that environment. The conclusion is simple: algorithms in large language environments should not directly learn raw logs. They should learn experience objects constructed through observation construction, action grounding, evidence verification, and update routing.

Figure 3 shows the core object. Raw logs inside a task micro-world ℓ_i become a learnable experience set $\mathcal{E}_i$ only after passing through observation, planning, action request, execution, environmental effect, evidence, and verification. Then the update router U decides whether the experience stays in the current context, enters memory and rules, or is further consolidated into skills, data, or weights.

Figure 4 states the same point more concretely. The algorithm is not consuming a raw chat transcript from left to right. It is consuming a constructed interface. The runtime determines what the model sees. The internal task environment helps the model plan and predict. The action-grounding layer turns requests into real effects. The feedback-evidence layer judges whether the result is reliable. The update router decides whether the experience affects current context, project memory, skills, data, or weights.

In traditional reinforcement learning, we often ask how to learn a policy from state to action in a given environment so that long-term reward is higher. At the highest level, RL in large language environments keeps the same spirit: experience should improve future behavior. The difference appears at the next level. Traditional formulations usually fix the environment, state, action, reward, and update target before optimizing the policy. In large language environments, those interfaces themselves must be constructed and calibrated. We can first divide the interface into the following elements:

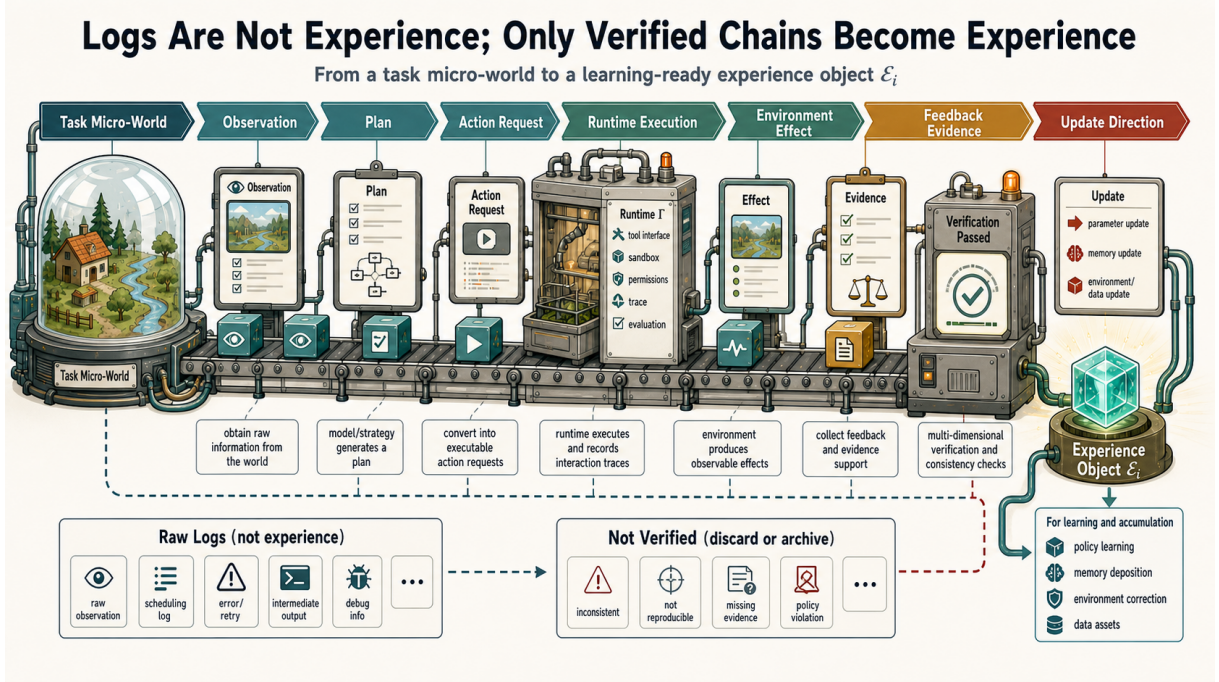


Figure 3: Learning objects in large language environments

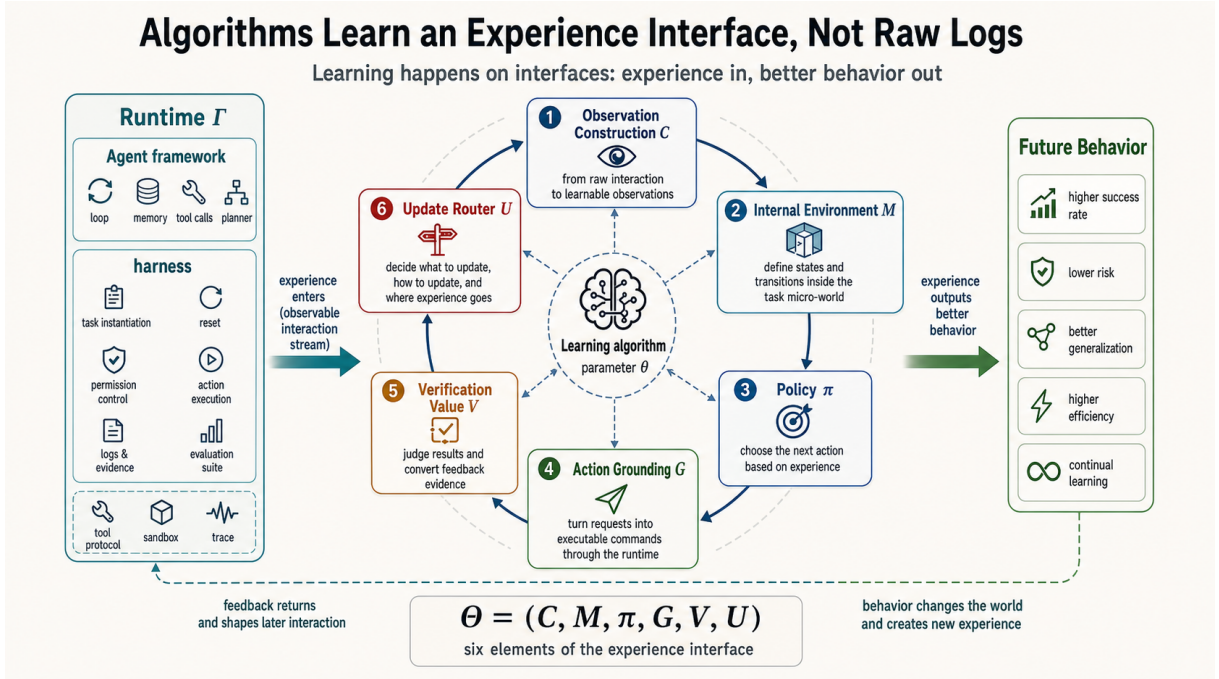


Figure 4: From task logs to updateable experience

$$\Theta = (C, M, \pi, G, V, U)$$

Here C constructs observations from history, tool outputs, memories, and rules. M is the internal task environment used for long thinking, planning, and self-verification. π proposes internal reasoning actions and external action requests. G grounds external requests into real actions. V broadly denotes verifiers, reward or value models, and preference judges that convert feedback evidence into verification, reward, preference, or constraint. U decides whether experience should be written to context, memory, skills, data, or weights.

Traditional reinforcement learning mainly asks how to learn π in a given environment. RL in large language environments must also ask how C, M, G, V , and U are learned and governed together. The harder part is that in real systems M, π , and V are often not three fully separate modules. They may be the same large model acting in different roles. Updating the policy may therefore also change the internal task environment and value judgment.

If we temporarily include continual learning in this notation, the experience accumulated in one task micro-world can be denoted by $\mathcal{E}_i$. But we should not rush to express the goal as a strict objective function. Future behavior improvement, update risk, verification cost, and experience transferability are not yet easy to capture in one reliable scalar. A safer statement is that continual learning studies which experiences in $\mathcal{E}_i$ apply only to the current micro-world, which can enter project memory or skill libraries, and which need stronger verification before entering data assets or weights.

The formalization is therefore a research direction rather than a final objective function. It shifts the question from “how do we maximize the current task score” to “how do we generate learnable experience, interpret evidence, and control update scope.” A task that succeeds while polluting long-term memory has not really been learned well. A task that fails but leaves reliable evidence, failure attribution, and a reversible rule may improve future tasks of the same kind.

Return to the coding-agent example. A learnable experience should at least record which repository micro-world the task came from, what files and errors the model observed, what internal plan it formed, what edit request it proposed, which files the runtime actually changed, what tests or review comments provided evidence, how strong the verification was, and where the experience was finally written. The notation can be complex, but the point is simple: an experience object must put task boundary, representation, internal task environment, plan, action, external effect, evidence, verification, and update into one chain.

Several concepts are easy to confuse here. Feedback is the raw signal after action: an error stack, test result, page state, human comment, or safety alert. Evidence is feedback with source, scope, confidence, and risk labels. An anchor is a relatively independent mechanism that can calibrate evidence, such as a compiler, unit test, real page state, formal rule, human expert, private evaluation, or institutional rule. Verification uses anchors to check whether a candidate action, result, or trajectory is reliable. Reward is the result of projecting evidence into an optimizable signal: a scalar score, preference comparison, constraint, reject-update marker, or training example.

In an ideal reinforcement learning setup, if the environment, goal, and reward function are clearly defined, scalar reward can carry enough information. This is not conceptually inconsistent with the reward-is-enough thesis (Silver et al., 2021). This essay is not arguing against reward maximization. It argues that in the engineering environments of LLM agents, reward should usually be the product of evidence interpretation, calibration, and routing. A compiler failure tells you a syntax location. A unit-test failure tells you a behavioral deviation. A review comment tells you a project convention. A user correction tells you a preference or misunderstanding. A safety alert tells you that a path must not be learned. Compressing these signals into one score too early discards the most useful learning structure.



Figure 5: Feedback first becomes evidence, then reward, preference, or constraint

The point of Figure 5 is that reward as a format is not the problem. The problem is letting reward come before evidence is understood. Work on process supervision and verifiers already shows that intermediate steps, final answers, tests, and graders carry different strengths of evidence [5]. The algorithmic goal in large language environments is not only to maximize the current task score. It is to learn three things: generate learnable experience at the representation layer, judge evidence and credit assignment at the interaction layer, and decide reward/value judgment and update routing at the coupling layer.

Where experience should be written is therefore not a post-processing detail. It is part of the learning objective. The U in $\Theta = (C, M, \pi, G, V, U)$ is not a normal logging system. It is the update router for continual reinforcement learning. It decides whether an experience should remain in the current context, enter short-term memory, become a project rule or skill candidate, become training data, or eventually affect model weights. Without U , reinforcement learning collapses back into current-task scoring. With U , the system begins to answer how reliable experience from one micro-world can improve future micro-worlds.

This also explains why continual reinforcement learning is not an optional enhancement. Stronger models help, but stronger models do not automatically solve experience governance. The stronger a model is, the more it can act; the more it can act, the larger the blast radius of wrong experience. If a strong model records public-benchmark tricks as general capability, writes one-off user preferences as long-term rules, packages dangerous tool use into a skill, or recursively trains on low-quality data it generated itself, the damage increases. Research on model collapse reminds us that the issue is not that synthetic data is always unusable. The issue is that recursively generated data without provenance, real-data mixing, diversity preservation, and evaluation isolation can amplify the wrong distribution over time [6].

Another reason is that much important future information is not in pretraining data. Enterprise codebases, long-term user preferences, real-time web pages, private tools, project history, and organizational processes are all obtained after deployment through interaction. Agents must learn while being used. The question is not whether they can adapt in the short term, but whether that adaptation can be verified, abstracted, reused, and revoked.

This leads to a key principle: weights should come last. Weight updates are not forbidden, but they have the highest evidence threshold, the highest rollback cost, and the widest impact scope. If a wrong experience stays only in the current context, it disappears after the task. If it enters long-term memory, it affects related future tasks. If it becomes a skill, it may be reused automatically. If it enters training data or weights, it spreads more broadly and becomes harder to undo.

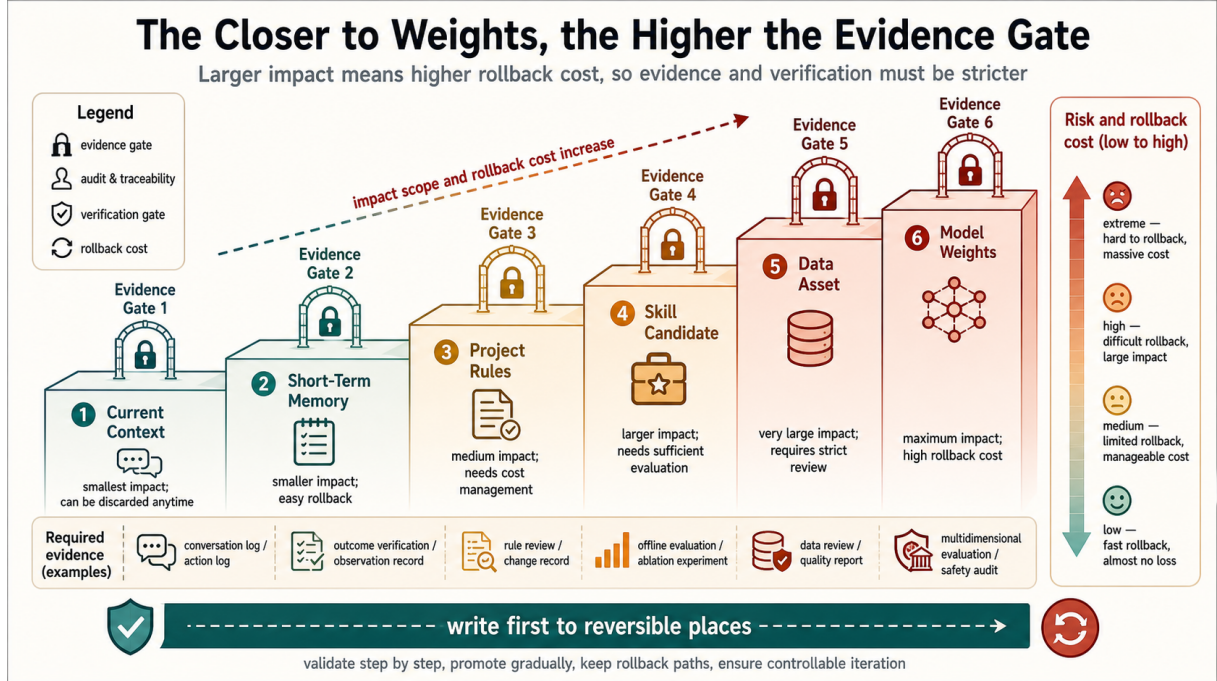


Figure 6: Experience update routing: the closer to weights, the higher the evidence threshold

Figure 6 shows the risk gradient of update carriers. The closer experience gets to model weights, the larger its impact and the harder it is to roll back, so the verification threshold must be higher. The earlier discussion asked whether evidence is trustworthy. Update routing asks how trustworthy it must be before it can enter different carriers. Without this route, an agent will confuse what happened with what should be learned.

Update hint: write first to reversible places. A new experience can first enter the current context, short-term memory, project rules, or skill candidates. Only after cross-task reproduction, independent verification, and safety checks should it be consolidated as a data asset or weight update. For large language environments, continual learning is first an update-routing problem and only then a parameter-update problem.

The more precise statement is this: the high-level goal is still future return or future behavior improvement. The essential difference is that traditional settings usually optimize a policy inside a fixed environment, while large language environments must also optimize the mechanisms of experience generation, evidence interpretation, and update routing. A system that scores higher on the current task but writes one-off preferences into long-term memory, treats a weak model score as hard reward, or consolidates incorrect tool use into a skill has not learned well. Conversely, a system may fail one task but preserve the right evidence, complete failure attribution, and update a reversible project rule, thereby improving future tasks.

Figure 7 summarizes the target as a coordinate diagram. The horizontal axis is not simply environment complexity. It is the clarity of the environment definition: clear simulators and fixed rules on the left, open and ambiguous real tools, web pages, and organizational processes

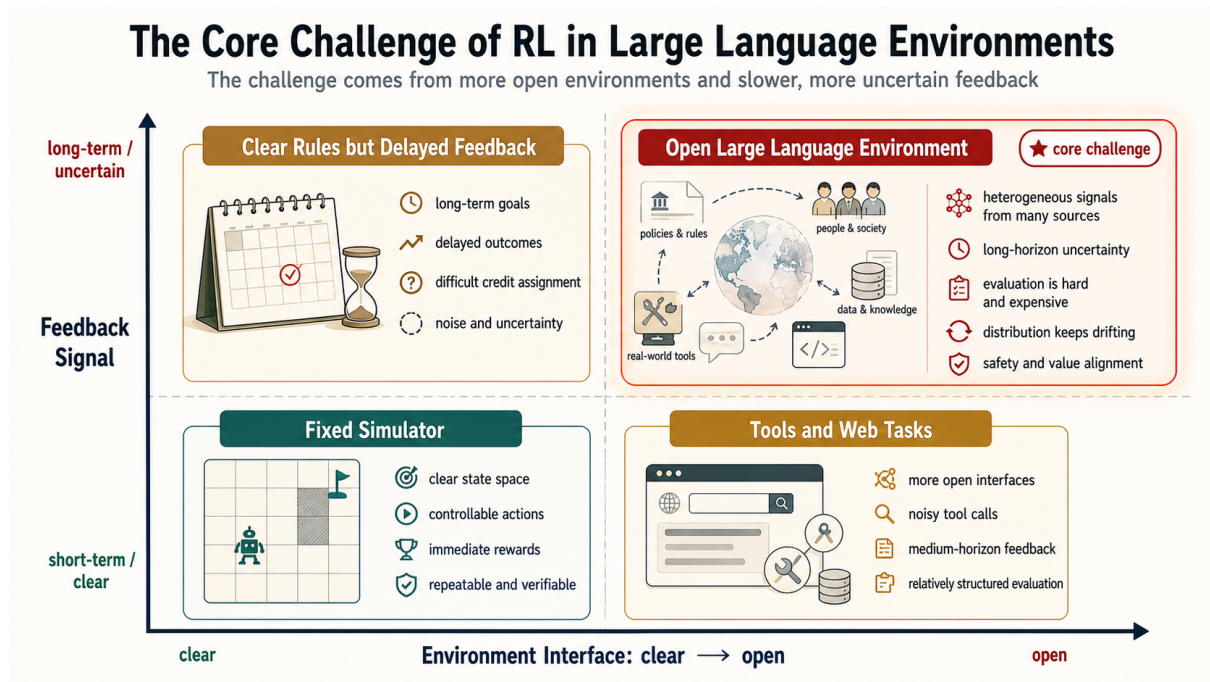


Figure 7: Core challenge and target of RL in large language environments

on the right. The vertical axis is not merely verification difficulty. It is the time scale and uncertainty of feedback signals: clear short-term signals such as win/loss, tests, and immediate reward at the bottom; more uncertain long-term effects, safety, and human values at the top. The goal of RL in large language environments is not to push every problem to the upper right. It is to construct learnable experience interfaces for open scenarios of that kind.

Hint: find reliable anchors first. When training coding agents, prioritize compilers, tests, static checks, code review, and bug reproduction. When training web and interface agents, record page state, target elements, before/after click states, network results, and completion evidence. When training office agents, include permissions, approvals, human confirmation, organizational rules, and the scope of preferences in feedback. When training data-analysis agents, preserve data versions, cleaning steps, chart outputs, statistical assumptions, and reproducible experiments. Different domains require different trajectories, verification budgets, and update routes.

5 Scaling: The Continual Experience Flywheel

This section only discusses experience scaling inside large language environments, not experience scaling for physical-world robotics. The physical world also requires learning from experience, but it involves sensors, physical dynamics, body control, hardware wear, and real-world safety constraints. Large language environments have different core objects: language context, tool calls, file state, web state, memory, verifiers, human feedback, and organizational permissions.

When people discuss scaling, the familiar path is pretraining-era scaling: more data, larger models, and more compute. That path remains important, but it mainly answers how static corpora become priors inside model parameters. Scaling in large language environments asks another question: how can experience produced by real language and tool interaction move from one-off task traces into a source of capability that is continually produced, verified, reused, and governed?

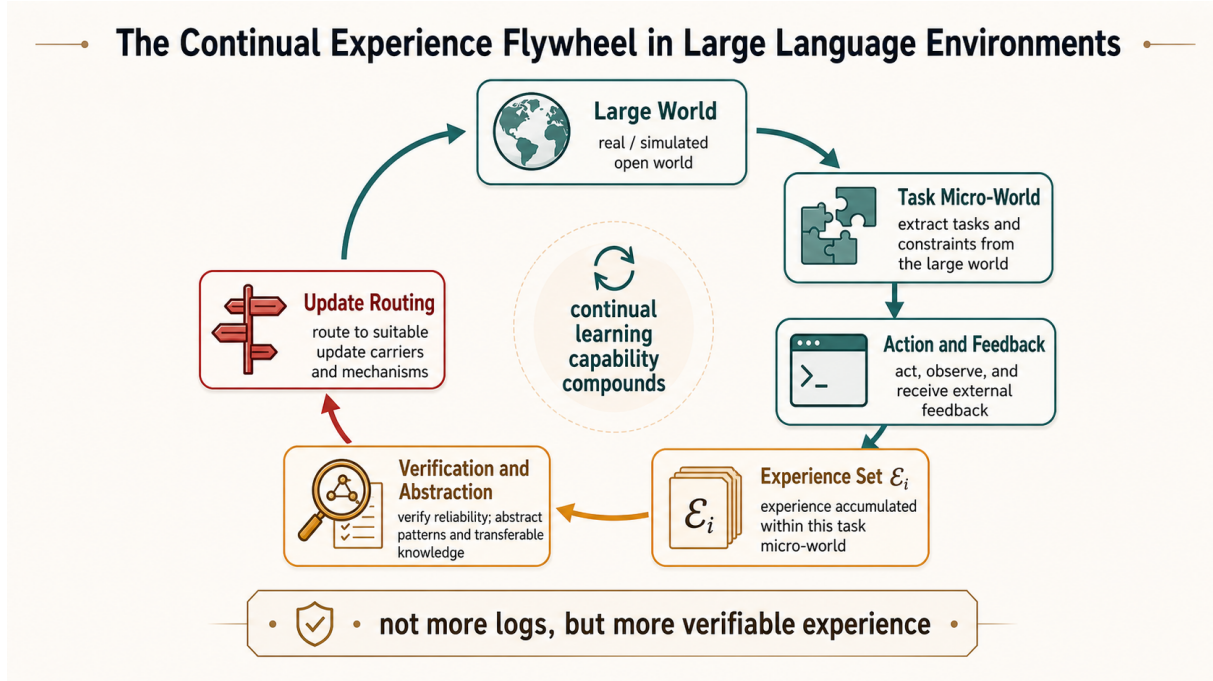


Figure 8: Experience scaling flywheel in large language environments

Figure 8 defines experience scaling as a loop. The large world continually instantiates task micro-worlds. The agent observes, thinks, acts, and receives feedback inside a micro-world. The system turns trajectories into experience objects, verifies and attributes them, then uses update routing to decide where they should be written back. After write-back, the large world affects how the next set of micro-worlds is constructed. The scaling target is not more logs. It is a loop that can steadily produce more verified, transferable, and reversible experience.

In the language of large worlds and micro-worlds, the input to the flywheel is the experience set $\mathcal{E}_i$ from each task micro-world. The output is not to mix all trajectories into one training pool. It is to route experience to the right place. Local experience stays in the local micro-world. Project experience enters project memory or rules. Transferable skills enter a skill library. Cross-domain stable experience may enter data assets or weights. Scaling continual reinforcement learning is not about repeatedly scoring higher on one task. It is about using reliable experience from many micro-worlds to improve how the large world is constructed.

This means scaling cannot be measured only by trajectory count. A system may produce one million tool-call logs per day. If most of them lack reliable verification, clear attribution, source records, reuse paths, or revocation mechanisms, it is not experience scaling in a large language environment. It is log scaling. True experience scaling requires at least five forms of scaling.

First, micro-world generation must scale. Agents should not learn only from a few fixed tasks. The large world should continuously generate bounded, goal-directed micro-worlds with tools and feedback: code, web, office, data analysis, multi-agent collaboration, long-running projects, private tools, and human organizational processes. The key is not to pile up task lists, but to make each task instance explicit about what it inherits from the large world, what it opens temporarily, and what it may write back after completion.

Second, experience objects must scale. Agents can attempt more actions, but those attempts must be recorded as learnable experience objects. The point is not merely to let the model try more often. It is to decompose each attempt into analyzable layers: what it saw, how it planned internally, what action request it made, what the runtime executed, what changed in the external environment, which step the feedback points to, and where the experience was finally written.

Third, verification must scale. Experience scaling ultimately depends on the supply of verifiable experience. Code tests, compilers, web states, user review, institutional rules, and private evaluations are all anchors, but they have different costs, coverage, and reliability. The key is not to hand all feedback to one model judge. It is to build a verification budget: cheap verification covers most everyday trajectories, expensive verification samples high-risk or high-value experience, and hard verification decides whether an experience can enter long-term memory, skills, or data assets.

Fourth, abstraction and transfer must scale. A single experience has limited value. Compound returns come when experience is abstracted into transferable rules, memories, cases, and skills. A data-analysis agent that only remembers the answer to one task is not very useful. It becomes useful when it abstracts dirty-data checks, modeling-error diagnosis paths, or report-generation conventions. The difficulty is distinguishing local regularities from general ones: success in a micro-world is not automatically knowledge in the large world.

Fifth, governance and write-back must scale. Once experience can be reused, it needs provenance, permissions, safety status, and revocation mechanisms. Scaling is not pushing every successful experience into weights. It is putting experience in the right place. Low-risk experience can enter short-term memory. Stable experience can enter project rules. High-value experience can enter skill candidates. Cross-task-verified experience can enter data assets. Weight updates should come last.

Continual reinforcement learning fits inside this flywheel. One-time post-training can make a model stronger on a class of tasks, but deployed agents will keep encountering new codebases, new web pages, new user preferences, new tool versions, new organizational rules, and new safety boundaries. The environment changes, tasks change, and feedback sources change. If learning only happens offline, an agent cannot truly compound experience after deployment. Continual reinforcement learning does not mean weights must change at every moment. It means context, memory, rules, skills, data, and weights should sit in a multi-timescale update system.

One viable direction is to split continual learning across timescales. Current context handles minute-level adaptation. Short-term memory handles task-level adaptation. Project rules and skills handle week- or month-level reuse. Data assets and weight updates handle slower and stricter capability internalization. The later the carrier, the higher the verification threshold and rollback cost.

Hint: the future object to collect is experience, not more chat logs. An interaction record should at least include the source and boundary of the task micro-world, visible state, internal-plan summary, actual actions, environmental changes, feedback evidence, verification strength, risk labels, and final update destination. Code experience should preserve patches, tests, and error stacks. Web experience should preserve target elements, before/after page states, and completion evidence. Office experience should preserve permissions, approvals, human confirmation, and preference scope. Collection must then add deduplication, desensitization, permission filtering, contamination detection, failure attribution, replay packaging, and revocation indexes. Only then can post-deployment interaction data move from something that merely happened into experience that can train, be audited, and be reused.

Future research should converge around the same formalization. We do not need to scatter algorithmic questions into many disconnected directions. Following the experience object and task-experience-set view above, the core algorithmic problems map to the same three layers.

First, representation-layer algorithms must handle large and ambiguous state, observation, and action spaces. They should learn not only how to fill the context window, but which history to preserve, which parts to summarize, which evidence must remain intact, and which internal

variables represent the real control state of the current task. For math and proof tasks, they must represent internal symbolic states and candidate proof spaces. For code and web tasks, they must represent external runtime state and grounded actions.

Second, interaction-layer algorithms must handle internal and external interaction together. Command-line agents, browser agents, and office agents can all complete complex external actions, but training still needs to separate internal plans, action requests, runtime execution, and external effects. Math and proof tasks also need to separate internal reasoning steps, self-checks, and final answers. Otherwise the system only knows that a task failed. It does not know whether to change observation, the internal task environment, the plan, tool parameters, permissions, or the runtime. Work on action decomposition has shown that this route can make the action space of language agents more trainable [7].

Third, coupling-layer algorithms must treat policy, internal task environment, and value judgment together. The algorithm must decide when to trust tests, when to trust human feedback, when to call expensive verifiers, and when to reject updates because evidence is too weak. It must also decide whether an experience should update the policy, internal environment model, value/reward judgment, memory, skill, or data asset. Continual reinforcement learning should not be equated with continual weight updates. One update may change the model's behavior, imagined environment, and self-evaluation standard at the same time. This is a multi-timescale control problem under coupled model roles.

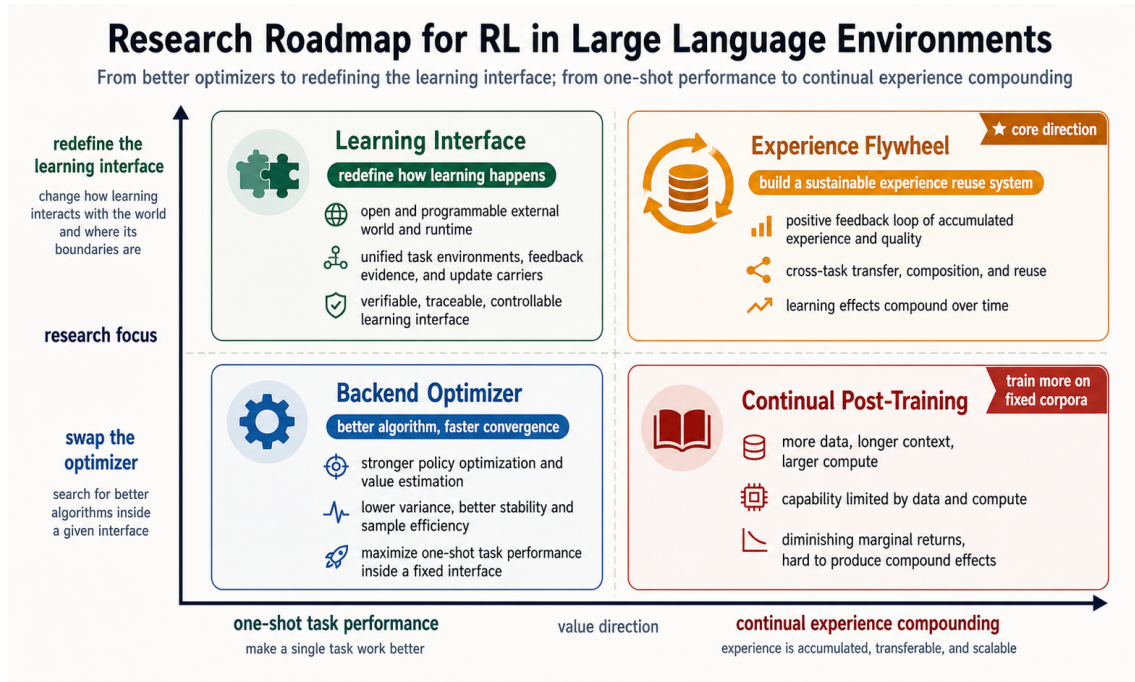


Figure 9: Research roadmap for reinforcement learning in large language environments

Figure 9 places these directions in a coordinate system. The horizontal axis moves from one-shot task performance toward continual experience compounding. The vertical axis moves from swapping backend optimizers toward redefining the front-end learning interface. The core direction is not to make one optimizer more complex. It is to make the optimizer consume the right experience objects: state and action representations, internal-external interaction, evidence calibration, credit assignment, and continual update routing.

6 Conclusion: From One-Shot Capability to Continual Experience

If reinforcement learning is part of the answer, the real question is not how to wrap PPO, GRPO, or another preference-optimization algorithm around agent logs. The question is what environment LLM agents are learning in, and how actions, feedback, and verification inside one task micro-world can become reusable, reversible, and auditable experience in the larger world.

This thread ties the essay together. A large language environment is not a single external simulator. It is a learning field composed of the external world, runtime, internal task environment, feedback evidence, and update carriers. Each task is instantiated as a micro-world from this larger world. The agent observes local state, performs internal planning, issues action requests, waits for runtime grounding, and receives feedback evidence. Continual reinforcement learning must decide which experiences are valid only in the current micro-world, which can be abstracted into rules, memories, skills, or data, and which evidence is strong enough to affect weights.

RL in large language environments therefore does not reject the traditional goal of cumulative return. It reopens interfaces that were often fixed in the traditional setup. States and actions need representation. Internal and external environments need separation. Feedback must first become evidence. Reward needs calibrated anchors. Updates need routing and revocation. Only then does future behavior improvement mean more than higher current-task score: it means more future task micro-worlds are constructed better, verified better, and less polluted by wrong experience.

This is the key move from answering well to accumulating experience. A model can perform strongly on a one-shot task while still failing to learn continually. It can also fail one task, preserve the right evidence, complete failure attribution, update the right carrier, and thereby genuinely improve on future tasks of the same kind. Experience scaling in the agent era is not about collecting more logs or updating weights more often. It is about building a reliable fly-wheel inside large language environments: micro-worlds produce experience, the large world absorbs experience, and new micro-worlds become easier to understand, act in, and verify.

That is the real problem RL in large language environments should solve. It is not a slogan about autonomous self-improvement, nor a mechanical application of classical RL to large models. It is the scientific and engineering problem of making agent improvement from verifiable experience recordable, testable, auditable, and falsifiable.

The tool-interaction and evaluation context includes ReAct [8], SWE-agent [9], SWE-bench [10], WebArena [11], and OSWorld [12]. These works motivate concrete interfaces and benchmarks without validating the complete framework proposed here. The broader reward-centered perspective is developed in Reward Is Enough [13].

References

- [1] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018. URL <http://incompleteideas.net/book/RLbook2020.pdf>.
- [2] David Silver and Richard S. Sutton. Welcome to the Era of Experience. Preprint chapter, 2025. URL <https://storage.googleapis.com/deepmind-media/Era-of-Experience%20/The%20Era%20of%20Experience%20Paper.pdf>.
- [3] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, et al. DeepSeek-Math: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv preprint arXiv:2402.03300, 2024. URL <https://arxiv.org/abs/2402.03300>.
- [4] LangChain. Frameworks, Runtimes, and Harnesses. Technical documentation, 2026. URL <https://docs.langchain.com/oss/python/concepts/products>. Accessed 2026-09-06.

- [5] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, et al. Let’s Verify Step by Step. arXiv preprint arXiv:2305.20050, 2023. URL <https://arxiv.org/abs/2305.20050>.
- [6] Ilia Shumailov, Zakhar Shumaylov, Yiren Zhao, Nicolas Papernot, Ross Anderson, and Yarin Gal. AI models collapse when trained on recursively generated data. *Nature*, 631(8022):755–759, 2024. doi: 10.1038/s41586-024-07566-y. URL <https://www.nature.com/articles/s41586-024-07566-y>.
- [7] Muning Wen, Ziyu Wan, Jun Wang, Weinan Zhang, and Ying Wen. Reinforcing LLM Agents via Policy Optimization with Action Decomposition. *Advances in Neural Information Processing Systems*, 37:103774–103805, 2024. doi: 10.52202/079017-3297. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/bc09efb501c801ed92e181e26a885c2d-Abstract-Conference.html.
- [8] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv preprint arXiv:2210.03629, 2022. URL <https://arxiv.org/abs/2210.03629>.
- [9] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. arXiv preprint arXiv:2405.15793, 2024. URL <https://arxiv.org/abs/2405.15793>.
- [10] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv preprint arXiv:2310.06770, 2023. URL <https://arxiv.org/abs/2310.06770>.
- [11] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, et al. WebArena: A Realistic Web Environment for Building Autonomous Agents. arXiv preprint arXiv:2307.13854, 2023. URL <https://arxiv.org/abs/2307.13854>.
- [12] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, et al. OS-World: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. arXiv preprint arXiv:2404.07972, 2024. URL <https://arxiv.org/abs/2404.07972>.
- [13] David Silver, Satinder Singh, Doina Precup, and Richard S. Sutton. Reward Is Enough. *Artificial Intelligence*, 299:103535, 2021. doi: 10.1016/j.artint.2021.103535. URL <https://doi.org/10.1016/j.artint.2021.103535>.